DSCI 519 · Homework 2

Support-Ticket Triage: Nested JSON, Regex, and an Evaluated LLM Step

Covers: Session 2 (Chapters 4 and 5). Points: 100. Submit: this notebook with outputs, the PDF made by the last cell, and `hw2_predictions.csv`.

The situation

LedgerLoop (a fictional invoicing and payments SaaS for small businesses) receives support tickets through email, chat, and a web form. The support manager wants every ticket automatically tagged with category, urgency, and sentiment, plus any invoice numbers and phone numbers pulled out of the message so agents can act faster. You have 78 tickets as exported from the ticketing API (nested JSON), and 20 of them are labeled (your development set). Your instructor holds the labels for the other 58 and will score your predictions on them.

Rubric

Part	What	Points
A	Flatten the nested JSON into a clean ticket table	12
B	Regex extraction of invoice IDs and phones, test-first	20
C	LLM classification: at least three prompt versions, evaluated on the dev set, error analysis	33
D	Consistency and cost analysis	15
E	Predictions file (scored on hidden labels) and a recommendation memo	20

Part E scoring: 10 points for the predictions file, scored on the hidden test set (average of category, urgency, and sentiment accuracy; 80% or better earns all 10; a missing or malformed file earns 0), and 10 points for the memo.

In [1]:
```
#@title Setup: install packages and the course helper (run me first)
# code hidden in this copy
```

```
llmkit 1.1.0 is ready.
```

Choose your AI provider

Everything in this course works with any of the providers below. Pick one, make sure its API key is stored in Colab Secrets (click the key icon in the left sidebar, add the secret name shown, and switch on *Notebook access*), and run the cell. You never paste a key into the notebook itself.

`PROVIDER`	Secret name	Cost notes (October 2026, verify before use)	Suggested `MIN_INTERVAL`
`"groq"`	`GROQ_API_KEY`	Free tier, no credit card, generous daily limits	2
`"google"`	`GOOGLE_API_KEY`	Free tier for Flash models (low daily quota; your data may be used for training)	6
`"openai"`	`OPENAI_API_KEY`	Paid, prepaid credit; small models cost cents for this course	0
`"anthropic"`	`ANTHROPIC_API_KEY`	Paid, prepaid credit; Haiku is the low-cost tier	0
`"openrouter"`	`OPENROUTER_API_KEY`	Many models behind one key; some `:free` models	3
`"mistral"`	`MISTRAL_API_KEY`	Free experimentation tier available	2
`"mock"`	none	Offline fake model: returns placeholders, useful only to test that code runs	0

`MODEL = None` uses the provider's default from `llmkit.PROVIDERS`. Model names change frequently; if you get a "model not found" error, run `llm.list_models()` and copy a current name into `MODEL`.

In [22]:
```
from llmkit import LLM, get_secret, estimate_tokens

PROVIDER = "google"    # "groq" | "google" | "openai" | "anthropic" | "openrouter" | "mistral" | "mock"
MODEL = None           # None = provider default, or a specific model id string
MIN_INTERVAL = 6.0     # seconds between calls (free tier allows 15 requests per minute)

llm = LLM(PROVIDER, model=MODEL, min_interval=MIN_INTERVAL, max_retries=6, cache=True)
print(llm)
```

```
LLM(provider='google', model='gemini-3.5-flash-lite')
```

In [23]:
```
#@title Get the data files
# code hidden in this copy
```

```
Files present: ['ledgerloop_dev_labels.csv', 'ledgerloop_tickets.json']
```

Part A. Flatten the tickets (12 points)

Load `ledgerloop_tickets.json` and produce a DataFrame `tickets` with exactly these columns: `ticket_id, received_at, channel, account_id, company, plan, contact_name, contact_email, contact_phone_e164, currency, autopay, subject, body, tags`

- `contact_phone_e164`: the CRM contact phone (`account.contact.phone`) normalized to `+1XXXXXXXXXX`; empty string if absent.
- `tags`: a comma-separated string.
- Use `pd.json_normalize` or `.get()` chains (your choice); some contacts have no phone, so handle missing keys.

Look at a raw record first: understand the shape before you prompt a companion.

In [24]:
```
import json, re, pandas as pd, numpy as np
raw = json.load(open("hw2/ledgerloop_tickets.json"))
dev = pd.read_csv("hw2/ledgerloop_dev_labels.csv", dtype=str).fillna("")
print(len(raw), "tickets;", len(dev), "labeled for development")
print(json.dumps(raw[0], indent=2))
```

```
78 tickets; 20 labeled for development
{
  "ticket_id": "T-1001",
  "received_at": "2025-03-13T11:14:00-05:00",
  "channel": "email",
  "account": {
    "account_id": "A-28043",
    "company": "Sunrise Daycare",
    "plan": "Standard",
    "contact": {
      "name": "Mark Holloway",
      "email": "mark@sunrisedaycare.com"
    },
    "billing": {
      "currency": "USD",
      "autopay": false
    }
  },
  "message": {
    "subject": "Sync with Google Sheets would be nice",
    "body": "Would love a native Google Sheets sync for our weekly revenue tracking. Currently we export by hand
```

```
every Monday. Keep up the good work!"
  },
  "tags": [
    "web",
    "trial"
  ]
}
```

In [25]:

```
# Part A: build `tickets`

# Normalize a phone to +1XXXXXXXXXX, or "" if missing or not 10 digits
def to_e164(phone):
    digits = re.sub(r"\D", "", phone or "")
    if len(digits) == 11 and digits.startswith("1"):
        digits = digits[1:]
    return "+1" + digits if len(digits) == 10 else ""

# Flatten each ticket with safe .get() chains
rows = []
for t in raw:
    account = t.get("account") or {}
    contact = account.get("contact") or {}
    billing = account.get("billing") or {}
    message = t.get("message") or {}
    rows.append({
        "ticket_id": t.get("ticket_id"),
        "received_at": t.get("received_at"),
        "channel": t.get("channel"),
        "account_id": account.get("account_id"),
        "company": account.get("company"),
        "plan": account.get("plan"),
        "contact_name": contact.get("name"),
        "contact_email": contact.get("email"),
        "contact_phone_e164": to_e164(contact.get("phone")),
        "currency": billing.get("currency"),
        "autopay": billing.get("autopay"),
        "subject": message.get("subject"),
        "body": message.get("body"),
        "tags": ", ".join(t.get("tags") or []),
    })

# Build the table with the required column order
columns = ["ticket_id", "received_at", "channel", "account_id", "company", "plan", "contact_name",
           "contact_email", "contact_phone_e164", "currency", "autopay", "subject", "body", "tags"]
tickets = pd.DataFrame(rows, columns=columns)

# Quick look at the result
print(tickets.shape)
print("Contacts with no phone:", (tickets.contact_phone_e164 == "").sum())
display(tickets.head())
```

```
(78, 14)
Contacts with no phone: 43
```

	ticket_id	received_at	channel	account_id	company	plan	contact_name	contact_email	contact_phone_e164	currency	autopay	subject	body	tags
0	T-1001	2025-03-13T11:14:00-05:00	email	A-28043	Sunrise Daycare	Standard	Mark Holloway	mark@sunrisedaycare.com		USD	False	Sync with Google Sheets would be nice	Would love a native Google Sheets sync for our...	web, trial
1	T-1002	2025-03-10T03:37:00-05:00	email	A-26372	Dakota Snow Pros	Basic	Amina Yusuf	amina@dakotasnowpros.com		USD	False	Charged twice this month	Hi, I was billed $49 twice on March 3 for our ...	
2	T-1003	2025-03-17T08:44:00-05:00	email	A-20682	Cedar Ridge Landscaping	Basic	Kevin Olson	kevin@cedarridgeland.com		USD	True	Close my account and delete data	Under our privacy policy obligations, please c...	web

3	T-1004	2025-03-21T20:57:00-05:00	web_form	A-29063	Big Sioux Brewing	Pro	Ben Fischer	ben@bigsiouxbrewin.com		USD	True	What payment methods do you support?	Can our clients pay invoices by ACH, or only b...	priority-customer
4	T-1005	2025-03-30T08:10:00-05:00	web_form	A-25579	Lakeview Chiropractic	Basic	Sara Lindgren	sara@lakeviewchirop.com		USD	True	Reports not loading	The Aging Report spins forever and never loads...	annual

In [26]:
```
# Checks (do not edit)
assert "tickets" in globals(), "Build the DataFrame `tickets` in Part A first, then rerun this cell."
expected = ["ticket_id", "received_at", "channel", "account_id", "company", "plan", "contact_name",
"contact_email",
            "contact_phone_e164", "currency", "autopay", "subject", "body", "tags"]
assert list(tickets.columns) == expected, "column names or order differ"
assert len(tickets) == 78 and tickets.ticket_id.is_unique
assert tickets.contact_phone_e164.fillna("").map(lambda p: p == "" or bool(re.fullmatch(r"\+1\d{10}",
p))).all()
print("Part A checks passed")
```

```
Part A checks passed
```

Part B. Regex, test first (20 points)

Write `extract_invoice_ids(text) -> list[str]` and `extract_phones(text) -> list[str]` (both return sorted, unique normalized values), following the rules in the labeling guide below.

1. Tests first. Starter tests are given; add at least four more of your own, including tricky non-matches.
2. Use a companion to draft the patterns ("make these tests pass; explain each part of the pattern").
3. Evaluate on the 20 dev tickets (exact match of the pipe-joined lists) and fix failures.

Labeling guide (this is how the gold labels were assigned)

Category (the primary issue; thank-you notes are categorized by the topic they mention):

- `Billing` : the customer's LedgerLoop subscription: charges, refunds, plans, discounts, receipts, payment methods, pricing questions.
- `Technical` : bugs, errors, outages, slowness, wrong calculations, or data problems inside LedgerLoop itself.
- `Account Access` : logins, passwords, two-factor, SSO, users, roles and permissions, account ownership, suspension.
- `Feature Request` : requests for new capabilities or changes to how the product works.
- `Integration` : third-party connections: QuickBooks, Stripe, Shopify, Zapier, Salesforce, HubSpot, bank feeds, webhooks, API limits, POS systems.
- `Cancellation` : cancelling, closing, not renewing, pausing instead of cancelling, exporting data before leaving, termination fees.

Urgency

- `Critical` : the customer's business is stopped right now, payments are failing broadly, security or privacy breach, or feared data loss.
- `High` : a major function is broken or blocked with a deadline within about two days, a financial error affecting the customer's clients, or repeated unresolved contact.
- `Medium` : a real problem with a workaround or without a near deadline, or a request that needs staff action (refund, account change, cancellation by a date).
- `Low` : questions, ideas, praise, minor issues, no time pressure.

Sentiment (the customer's tone, not the topic): `Negative` (frustrated, angry, disappointed, sarcastic), `Neutral` (factual or polite), `Positive` (praise, gratitude, enthusiasm).

Invoice IDs: invoice references in the body normalized to `INV-YYYY-NNNNN` (five-digit number). If the year is missing, use 2025. Not invoices: ticket numbers, confirmation codes such as `#CX-55120`, payout batch numbers, and amounts. Phones: US phone numbers in the body normalized to E.164 (`+1` followed by 10 digits).

In [27]:
```
LABEL_GUIDE = """
### Labeling guide (this is how the gold labels were assigned)
**Category** (the primary issue; thank-you notes are categorized by the topic they mention):
* `Billing`: the customer's LedgerLoop subscription: charges, refunds, plans, discounts, receipts, payment
methods, pricing questions.
* `Technical`: bugs, errors, outages, slowness, wrong calculations, or data problems inside LedgerLoop itself.
* `Account Access`: logins, passwords, two-factor, SSO, users, roles and permissions, account ownership,
suspension.
* `Feature Request`: requests for new capabilities or changes to how the product works.
* `Integration`: third-party connections: QuickBooks, Stripe, Shopify, Zapier, Salesforce, HubSpot, bank
feeds, webhooks, API limits, POS systems.
* `Cancellation`: cancelling, closing, not renewing, pausing instead of cancelling, exporting data before
leaving, termination fees.

**Urgency**
* `Critical`: the customer's business is stopped right now, payments are failing broadly, security or privacy
breach, or feared data loss.
* `High`: a major function is broken or blocked with a deadline within about two days, a financial error
affecting the customer's clients, or repeated unresolved contact.
* `Medium`: a real problem with a workaround or without a near deadline, or a request that needs staff action
(refund, account change, cancellation by a date).
* `Low`: questions, ideas, praise, minor issues, no time pressure.

**Sentiment** (the customer's tone, not the topic): `Negative` (frustrated, angry, disappointed, sarcastic),
`Neutral`
(factual or polite), `Positive` (praise, gratitude, enthusiasm).

**Invoice IDs**: invoice references in the body normalized to `INV-YYYY-NNNNN` (five-digit number). If the
year is missing,
use 2025. Not invoices: ticket numbers, confirmation codes such as `#CX-55120`, payout batch numbers, and
amounts.
**Phones**: US phone numbers in the body normalized to E.164 (`+1` followed by 10 digits).
"""
print("LABEL_GUIDE loaded: use it in your prompts.")
```

```
LABEL_GUIDE loaded: use it in your prompts.
```

In [28]:
```
INVOICE_TESTS = {
    "Invoice #2025-00417 shows both.": ["INV-2025-00417"],
    "INV 2025/00388 and INV 2025/00391 never showed up": ["INV-2025-00388", "INV-2025-00391"],
    "Invoice #00452 is an example.": ["INV-2025-00452"],
    "confirmation #CX-55120 was sent": [],
}
PHONE_TESTS = {
    "Call me at (605) 555-0142 please": ["+16055550142"],
    "Phone +1 712 555 0123.": ["+17125550123"],
    "payout batch 18 differs by $212.40": [],
}

def run_tests(fn, tests):
    ok = 0
    for text, want in tests.items():
        got = fn(text)
        ok += got == want
        print(("PASS " if got == want else "FAIL ") + f"{text[:55]!r:58} want={want} got={got}")
    print(f"{ok}/{len(tests)} passed\n")
```

In [29]:
```
# Add at least four tests of your own to INVOICE_TESTS and PHONE_TESTS, then write the functions.

# My own invoice tests (tricky matches and non-matches)
INVOICE_TESTS.update({
    "Please check inv-2025-00610 on our account": ["INV-2025-00610"],
    "Same issue as before (sample #2025-01022).": ["INV-2025-01022"],
    "invoice #2025-00733 and Invoice #2025-00733 again": ["INV-2025-00733"],
    "Resolved in ticket 4471, thanks": [],
    "Imported 2,000 rows but it stopped at 1,214": [],
    "We are a 501(c)(3) and paid $212.40": [],
    "Numbers restarted at 0001 after the update": [],
})

# My own phone tests (tricky matches and non-matches)
PHONE_TESTS.update({
    "Reach me on 1-605-555-0110 today": ["+16055550110"],
    "Call 605 555 0188 or 402.555.0199": ["+14025550199", "+16055550188"],
    "Office line 605-555-0177, thanks": ["+16055550177"],
    "Invoice #2025-00417 and INV 2025/00388": [],
    "Tracking number 60555501429931 is not a phone": [],
    "Imported 2,000 invoices, stopped at 1,214": [],
})

# Invoice pattern: "inv"/"invoice" or "#", optional 4-digit year with - or /, then exactly 5 digits
INVOICE_RE = re.compile(r"(?:\binv(?:oice)?[\s#:-]*|#)(?:(20\d{2})[-/])?(\d{5})(?!\d)", re.IGNORECASE)

def extract_invoice_ids(text):
    # Sorted, unique IDs as INV-YYYY-NNNNN (year defaults to 2025)
    found = {f"INV-{year or '2025'}-{num}" for year, num in INVOICE_RE.findall(text or "")}
    return sorted(found)

# Phone pattern: optional +1, optional (area code), 3 + 4 digits, no digit right before or after
PHONE_RE = re.compile(r"(?<!\d)(?:\+?1[\s.-]?)?\(?(\d{3})\)?[\s.-]?(\d{3})[\s.-]?(\d{4})(?!\d)")

def extract_phones(text):
    # Sorted, unique phones in E.164 format (+1 and 10 digits)
    found = {"+1" + "".join(parts) for parts in PHONE_RE.findall(text or "")}
    return sorted(found)

run_tests(extract_invoice_ids, INVOICE_TESTS)
run_tests(extract_phones, PHONE_TESTS)
```

```
PASS 'Invoice #2025-00417 shows both.'                          want=['INV-2025-00417'] got=['INV-2025-00417']
PASS 'INV 2025/00388 and INV 2025/00391 never showed up'        want=['INV-2025-00388', 'INV-2025-00391'] got=
['INV-2025-00388', 'INV-2025-00391']
PASS 'Invoice #00452 is an example.'                            want=['INV-2025-00452'] got=['INV-2025-00452']
PASS 'confirmation #CX-55120 was sent'                          want=[] got=[]
PASS 'Please check inv-2025-00610 on our account'               want=['INV-2025-00610'] got=['INV-2025-00610']
PASS 'Same issue as before (sample #2025-01022).'               want=['INV-2025-01022'] got=['INV-2025-01022']
PASS 'invoice #2025-00733 and Invoice #2025-00733 again'        want=['INV-2025-00733'] got=['INV-2025-00733']
PASS 'Resolved in ticket 4471, thanks'                          want=[] got=[]
PASS 'Imported 2,000 rows but it stopped at 1,214'              want=[] got=[]
PASS 'We are a 501(c)(3) and paid $212.40'                      want=[] got=[]
PASS 'Numbers restarted at 0001 after the update'               want=[] got=[]
11/11 passed

PASS 'Call me at (605) 555-0142 please'                         want=['+16055550142'] got=['+16055550142']
PASS 'Phone +1 712 555 0123.'                                   want=['+17125550123'] got=['+17125550123']
PASS 'payout batch 18 differs by $212.40'                       want=[] got=[]
PASS 'Reach me on 1-605-555-0110 today'                         want=['+16055550110'] got=['+16055550110']
PASS 'Call 605 555 0188 or 402.555.0199'                        want=['+14025550199', '+16055550188'] got=['+140
25550199', '+16055550188']
PASS 'Office line 605-555-0177, thanks'                         want=['+16055550177'] got=['+16055550177']
PASS 'Invoice #2025-00417 and INV 2025/00388'                   want=[] got=[]
PASS 'Tracking number 60555501429931 is not a phone'            want=[] got=[]
PASS 'Imported 2,000 invoices, stopped at 1,214'                want=[] got=[]
```

```
9/9 passed
```

In [30]:
```python
# Evaluate on the dev set (do not edit)
assert "tickets" in globals(), "Complete Part A (the `tickets` DataFrame) first, then rerun this cell."
d = tickets.merge(dev, on="ticket_id")
for col, fn in [("invoice_ids", extract_invoice_ids), ("body_phones", extract_phones)]:
    pred = d.body.map(lambda t: "|".join(fn(t)))
    print(f"{col}: {(pred == d[col]).mean():.0%} exact match on dev")
    misses = d[pred != d[col]]
    if len(misses):
        display(pd.DataFrame({"ticket_id": misses.ticket_id, "expected": misses[col], "predicted":
pred[misses.index]}))
```

```
invoice_ids: 100% exact match on dev
body_phones: 100% exact match on dev
```

Part C. LLM classification (33 points)

Build a classifier that returns `dict(category=..., urgency=..., sentiment=...)` using `llm.ask` (or `llm.chat`) and validation in Python (allowed labels only; anything else becomes `None`). Structured outputs with contracts come in Session 3; here, practice constraining and validating plain responses, for example by asking for a single JSON object (`llm.ask(..., json_mode=True)`) that you parse with `llmkit.parse_json_object`. Session 2, section 3.8 shows this pattern with two fields; here you need three.

Requirements:

1. At least three prompt versions (`v1`, `v2`, `v3`). Each must be a deliberate change motivated by the error analysis of the previous version (definitions, the label guide, few-shot examples that are *not* dev tickets, output format).
2. For each version, report dev-set accuracy for each field, the number of invalid outputs, and a confusion matrix for `urgency`.
3. Error analysis after each version: list the misclassified dev tickets and categorize the reasons.

> Data leakage warning. Do not paste dev tickets (or their labels) into your prompts as examples: your dev accuracy would no longer estimate performance on new tickets. Write your own examples.

In [31]:
```
CATEGORIES = ["Billing", "Technical", "Account Access", "Feature Request", "Integration", "Cancellation"]
URGENCY = ["Low", "Medium", "High", "Critical"]
SENTIMENT = ["Negative", "Neutral", "Positive"]

def validate_label(value, allowed):
    """Map a raw model string to one allowed label (case-insensitive), else None."""
    if not isinstance(value, str):
        return None
    v = value.strip().strip(".").lower()
    return next((a for a in allowed if a.lower() == v), None)

def evaluate(pred_df, label):
    d = dev.merge(pred_df, on="ticket_id", suffixes=("_gold", "_pred"))
    out = {}
    for f in ["category", "urgency", "sentiment"]:
        out[f] = round(float((d[f + "_gold"] == d[f + "_pred"]).mean()), 2)
        out[f + "_invalid"] = int(d[f + "_pred"].isna().sum())
    print(label, out)
    display(pd.crosstab(d.urgency_gold, d.urgency_pred.fillna("INVALID"), rownames=["gold"], colnames=
["pred"]))
    return d
```

In [32]:
```
# Version 1
import llmkit

SYSTEM_V1 = """You classify customer support tickets for LedgerLoop, an invoicing and payments software
company.
Return only a JSON object with exactly three keys:
"category": one of Billing, Technical, Account Access, Feature Request, Integration, Cancellation
"urgency": one of Low, Medium, High, Critical
"sentiment": one of Negative, Neutral, Positive"""

# Shared helper for every version: call the model, parse the JSON reply, validate each field
def classify(subject, body, system):
    raw = llm.ask(f"Subject: {subject}\n\nBody: {body}", system=system,
                  json_mode=True, max_tokens=200, temperature=0)
    try:
        obj = llmkit.parse_json_object(raw)
    except ValueError:
        obj = {}
    if not isinstance(obj, dict):
        obj = {}
    return dict(category=validate_label(obj.get("category"), CATEGORIES),
                urgency=validate_label(obj.get("urgency"), URGENCY),
                sentiment=validate_label(obj.get("sentiment"), SENTIMENT))

def classify_v1(subject, body):
    # call the model, parse, validate
    return classify(subject, body, SYSTEM_V1)

# Shared helper: list every wrong field with the ticket subject, for error analysis
def show_errors(d):
    d = d.merge(tickets[["ticket_id", "subject"]], on="ticket_id")
    rows = []
    for f in ["category", "urgency", "sentiment"]:
        for r in d[d[f + "_gold"] != d[f + "_pred"]].itertuples():
            rows.append({"ticket_id": r.ticket_id, "subject": r.subject, "field": f,
                         "gold": getattr(r, f + "_gold"), "pred": getattr(r, f + "_pred")})
    return pd.DataFrame(rows)

dev_tickets = tickets[tickets.ticket_id.isin(dev.ticket_id)]
pred_v1 = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_v1(r.subject, r.body)) for r in
dev_tickets.itertuples()])
d1 = evaluate(pred_v1, "v1")
display(show_errors(d1))
```

```
v1 {'category': 0.95, 'category_invalid': 0, 'urgency': 0.65, 'urgency_invalid': 0, 'sentiment': 0.9, 'sentiment_invalid': 0}
```

pred gold	Critical	High	Low	Medium
Critical	1	0	0	0
High	1	1	0	0
Low	0	0	9	1
Medium	0	3	2	2

	ticket_id	subject	field	gold	pred
0	T-1009	How do I add a team member?	category	Account Access	Technical
1	T-1002	Charged twice this month	urgency	Medium	High
2	T-1005	Reports not loading	urgency	Medium	Low
3	T-1007	Recurring invoices skipped	urgency	Medium	High
4	T-1008	Double charged AND locked out	urgency	High	Critical
5	T-1013	Who can see payroll invoices	urgency	Medium	Low
6	T-1015	Won't renew	urgency	Low	Medium
7	T-1017	Very disappointed, cancelling	urgency	Medium	High
8	T-1010	Calendar view of due dates	sentiment	Neutral	Positive
9	T-1012	Request: client-level payment terms	sentiment	Neutral	Positive

v1 error analysis. *Which tickets were wrong, and why? What will you change in v2?*

Results: category 95% (19/20), urgency 65% (13/20), sentiment 90% (18/20), 0 invalid outputs in all three fields. Urgency is the weakest field. In the confusion matrix, only 2 of the 7 Medium tickets were labeled correctly: 3 were labeled High and 2 were labeled Low. Overall, the model rated urgency one level too high on 5 tickets and one level too low on 2.

Misclassified tickets, grouped by reason:

Reason	Tickets	Gold → Predicted
Urgency too high when there was a workaround or no near deadline	T-1002 (duplicate charge refund), T-1007 (invoices already sent manually), T-1017 (cancel at end of billing period)	Medium → High
Urgency too high for a problem that is serious but not stopping the business	T-1008 (double charge and locked out)	High → Critical
Urgency too high for a cancellation with a date far away	T-1015 (won't renew on June 1)	Low → Medium
Urgency too low for a real problem or a request that needs staff action	T-1005 (report not loading), T-1013 (restrict invoice visibility)	Medium → Low
Category: a question about users and roles labeled as a bug	T-1009 (add a team member)	Account Access → Technical
Sentiment: polite feature ideas labeled as enthusiasm	T-1010 ("Just an idea"), T-1012 ("We'd use it daily")	Neutral → Positive

Why: the v1 prompt lists only the label names, with no definitions. The model does not know that a real problem with a workaround is Medium, that only a stopped business or a security breach is Critical, that users and roles belong under Account Access, or that a polite request is Neutral rather than Positive.

Change for v2: add the full labeling guide (`LABEL_GUIDE`) to the system prompt, so the model has the definition of every category, urgency level, and sentiment. The output format and everything else stay the same, so any change in accuracy comes from the definitions alone.

In [33]:
```python
# Version 2

# v2 prompt: same task and output format as v1, plus the full labeling guide
SYSTEM_V2 = f"""You classify customer support tickets for LedgerLoop, an invoicing and payments software
company.
Use the category, urgency, and sentiment definitions in this labeling guide:
{LABEL_GUIDE}
Return only a JSON object with exactly three keys:
"category": one of Billing, Technical, Account Access, Feature Request, Integration, Cancellation
"urgency": one of Low, Medium, High, Critical
"sentiment": one of Negative, Neutral, Positive"""

def classify_v2(subject, body):
    # call the model, parse, validate
    return classify(subject, body, SYSTEM_V2)

pred_v2 = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_v2(r.subject, r.body)) for r in
dev_tickets.itertuples()])
d2 = evaluate(pred_v2, "v2")
display(show_errors(d2))
```

```
v2 {'category': 0.85, 'category_invalid': 0, 'urgency': 0.75, 'urgency_invalid': 0, 'sentiment': 0.95, 'sentimen
t_invalid': 0}
```

pred gold	Critical	High	Low	Medium
Critical	1	0	0	0
High	0	2	0	0
Low	0	0	9	1
Medium	0	1	3	3

	ticket_id	subject	field	gold	pred
0	T-1013	Who can see payroll invoices	category	Feature Request	Account Access
1	T-1018	Amazing support team	category	Account Access	Billing
2	T-1020	Customer portal link expired	category	Technical	Feature Request
3	T-1005	Reports not loading	urgency	Medium	Low
4	T-1007	Recurring invoices skipped	urgency	Medium	High
5	T-1013	Who can see payroll invoices	urgency	Medium	Low
6	T-1015	Won't renew	urgency	Low	Medium
7	T-1020	Customer portal link expired	urgency	Medium	Low
8	T-1007	Recurring invoices skipped	sentiment	Negative	Neutral

v2 error analysis. ...

Change from v1: added the full labeling guide (`LABEL_GUIDE`) to the system prompt. The output format stayed the same.

Results: category 85% (17/20), urgency 75% (15/20), sentiment 95% (19/20), 0 invalid outputs in all three fields.

Field	v1	v2	Change
Category	95%	85%	−2 tickets
Urgency	65%	75%	+2 tickets
Sentiment	90%	95%	+1 ticket

Total errors went from 10 to 9. The guide fixed six v1 errors: T-1009 category (adding a team member is now Account Access), T-1002, T-1008 and T-1017 urgency (no longer rated too high), and T-1010 and T-1012 sentiment (polite ideas are now Neutral). In the urgency confusion matrix, High (2/2) and Critical (1/1) are now all correct, but only 3 of the 7 Medium tickets are correct: 3 were labeled Low and 1 High. The direction of the urgency errors flipped: v1 mostly rated urgency too high, v2 mostly rates it too low.

Misclassified tickets, grouped by reason:

Reason	Tickets	Gold → Predicted
Urgency too low: a real malfunction that the customer describes calmly ("not urgent", "can that be extended?")	T-1005 (report not loading), T-1020 (portal link expires), T-1013 (restrict invoice visibility)	Medium → Low
Urgency too high: the customer already has a workaround	T-1007 (invoices already sent manually)	Medium → High
Urgency too high: a cancellation whose date is months away	T-1015 (won't renew on June 1)	Low → Medium
Category: the word "admins" pulled it to Account Access, but it asks for a new capability	T-1013 (restrict invoice visibility)	Feature Request → Account Access
Category: a thank-you note categorized by the wrong topic	T-1018 (thanks for help with account setup)	Account Access → Billing
Category: a broken behavior labeled as a request because the customer asked for a change	T-1020 (portal link expires)	Technical → Feature Request
Sentiment: mild frustration missed	T-1007 (recurring invoices skipped)	Negative → Neutral

Why: the definitions help, but the model still struggles at the boundaries between labels: Medium versus Low when the customer sounds calm, Technical versus Feature Request, and which topic a thank-you note belongs to. The 2-ticket drop in category accuracy may partly be run-to-run randomness on a set of only 20 tickets; Part D measures how much labels change between identical runs.

Change for v3: keep the labeling guide and add (1) short decision rules for these boundaries and (2) a few example tickets that I wrote myself (not dev tickets) showing each boundary case, so the model sees how the rules apply.

In [34]:
```
# Version 3 (and more if you like)
# Version 3 (and more if you like)

# v3 additions: decision rules for the boundary cases found in v2
RULES_V3 = """Decision rules for hard cases:
1. Urgency, Medium vs Low: if a LedgerLoop function is broken or behaving wrongly, urgency is at least Medium,
   even when the customer sounds calm or says "not urgent". Use Low only for questions, ideas, praise,
   or small issues the customer calls minor.
2. Urgency, Medium vs High: if the customer already has a workaround and no deadline within about two days,
use Medium, not High.
3. Urgency for cancellations: a cancellation or non-renewal whose date is more than a month away, with no
other problem, is Low.
4. Category, Technical vs Feature Request: if existing behavior is wrong or broken, it is Technical, even if
the customer
   asks you to change it. Feature Request is only for something the product does not do today, including new
permission or visibility controls.
5. Category for thank-you notes: use the topic the help was about (for example, help with logging in is
Account Access).
6. Sentiment: mild frustration or complaint words ("annoying", "embarrassing", "tedious") are Negative. A
polite request with no praise is Neutral."""

# v3 additions: example tickets written by me (not dev tickets), one per rule
EXAMPLES_V3 = """Examples:
Subject: Statement totals off by a few cents
Body: The monthly statement total is a few cents off from the invoice lines. No rush, we can adjust by hand
for now.
{"category": "Technical", "urgency": "Medium", "sentiment": "Neutral"}

Subject: Reminder emails failed last night
Body: None of our scheduled reminders went out last night. I resent them by hand this morning, but please make
```

```
sure the next run works.
{"category": "Technical", "urgency": "Medium", "sentiment": "Negative"}

Subject: Ending our plan in September
Body: We will not continue after our term ends in September. Please note this on our account.
{"category": "Cancellation", "urgency": "Low", "sentiment": "Neutral"}

Subject: Option to hide amounts from staff
Body: Could you add a setting so only managers can see invoice amounts? We can't do that today.
{"category": "Feature Request", "urgency": "Low", "sentiment": "Neutral"}

Subject: Thanks for getting me back in
Body: Your agent helped me get back into my account after I was locked out. Great service!
{"category": "Account Access", "urgency": "Low", "sentiment": "Positive"}

Subject: Search shows deleted clients
Body: Searching for a client still shows records we deleted months ago. Annoying to scroll past them every
time.
{"category": "Technical", "urgency": "Medium", "sentiment": "Negative"}

Subject: Logo slightly blurry
Body: Small thing: our logo looks a bit blurry on printed invoices. Not a big deal.
{"category": "Technical", "urgency": "Low", "sentiment": "Neutral"}"""

# v3 prompt: v2 prompt + decision rules + my own examples
SYSTEM_V3 = f"""You classify customer support tickets for LedgerLoop, an invoicing and payments software
company.
Use the category, urgency, and sentiment definitions in this labeling guide:
{LABEL_GUIDE}
{RULES_V3}

{EXAMPLES_V3}

Return only a JSON object with exactly three keys:
"category": one of Billing, Technical, Account Access, Feature Request, Integration, Cancellation
"urgency": one of Low, Medium, High, Critical
"sentiment": one of Negative, Neutral, Positive"""

def classify_v3(subject, body):
    # call the model, parse, validate
    return classify(subject, body, SYSTEM_V3)

pred_v3 = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_v3(r.subject, r.body)) for r in
dev_tickets.itertuples()])
d3 = evaluate(pred_v3, "v3")
display(show_errors(d3))
```

```
v3 {'category': 0.9, 'category_invalid': 0, 'urgency': 0.85, 'urgency_invalid': 0, 'sentiment': 0.95, 'sentiment
_invalid': 0}
```

pred gold	Critical	High	Low	Medium
Critical	1	0	0	0
High	0	2	0	0
Low	0	0	9	1
Medium	0	0	2	5

	ticket_id	subject	field	gold	pred
0	T-1001	Sync with Google Sheets would be nice	category	Feature Request	Integration
1	T-1020	Customer portal link expired	category	Technical	Feature Request
2	T-1013	Who can see payroll invoices	urgency	Medium	Low
3	T-1015	Won't renew	urgency	Low	Medium

4	T-1020	Customer portal link expired	urgency	Medium	Low
5	T-1007	Recurring invoices skipped	sentiment	Negative	Neutral

v3 error analysis and final choice. *Which version will you deploy, and why? Is the improvement larger than what you would expect from run-to-run randomness (see Part D)?*

Change from v2: kept the labeling guide and added (1) six short decision rules, one for each type of v2 error, and (2) seven example tickets that I wrote myself (not dev tickets), each showing one rule with its correct JSON answer. The output format stayed the same.

Results: category 90% (18/20), urgency 85% (17/20), sentiment 95% (19/20), 0 invalid outputs in all three fields.

Field	v1	v2	v3
Category	95%	85%	90%
Urgency	65%	75%	85%
Sentiment	90%	95%	95%
Average of the three	83%	85%	90%
Total wrong fields	10	9	6

v3 fixed four v2 errors: T-1013 category (restricting invoice visibility is now Feature Request), T-1018 category (a thank-you note for account setup is now Account Access), T-1005 urgency (a broken report is now Medium even though the customer said "not urgent"), and T-1007 urgency (a problem with a workaround is now Medium, not High). In the urgency confusion matrix, High (2/2) and Critical (1/1) are all correct, Low is 9/10, and Medium improved from 3/7 in v2 to 5/7.

Remaining errors, grouped by reason:

Reason	Tickets	Gold → Predicted
Category: a request for a new connection to a tool not on the supported list was labeled Integration (new in v3)	T-1001 (Google Sheets sync)	Feature Request → Integration
Category and urgency: the customer asks "can that be extended?", so the model sees a request instead of a broken behavior (wrong in all three versions)	T-1020 (portal link expires)	Technical / Medium → Feature Request / Low
Urgency too low for a request that needs staff action	T-1013 (restrict invoice visibility)	Medium → Low
Urgency too high for a cancellation with a far-off date, despite rule 3	T-1015 (won't renew on June 1)	Low → Medium
Sentiment: mild frustration missed (wrong in v2 and v3)	T-1007 (recurring invoices skipped)	Negative → Neutral

T-1015 shows a limit of the input rather than the prompt: the model is never told when the ticket was received (March 3), so it cannot tell that June 1 is three months away. A possible future improvement is to include `received_at` in the message sent to the model.

Final choice: v3. It has the highest average accuracy (90%), the fewest wrong fields (6 versus 10 in v1), 0 invalid outputs, and the largest gain on urgency (65% → 85%), which is the field that matters most for routing tickets. I did not build a v4: with only 20 dev tickets, further changes would mostly fix individual dev tickets, which risks fitting the prompt to this set instead of to new tickets.

Is the improvement larger than run-to-run randomness? Yes. In Part D, v3 gave identical labels for all three fields on 20 of 20 dev tickets (100%) across two runs with the cache turned off, and both runs had exactly the same accuracy (category 90%, urgency 85%, sentiment 95%). With 0 tickets changing between identical runs, the v1 → v3 gain of 4 fewer wrong fields (urgency +4 tickets, 65% → 85%) comes from the prompt changes, not from

chance. One caution: the dev set has only 20 tickets, so each ticket is worth 5 percentage points, and the hidden test set may give somewhat different numbers.

Part D. Consistency and cost (15 points)

1. Test-retest: run your final classifier twice on the dev set with a client that has `cache=False`. Report the share of tickets where all three fields are identical across runs.
2. Cost: from the usage log, compute average input and output tokens per ticket for the final version. Look up your model's current price and project the monthly cost for 20,000 tickets per month. On a free tier, also state the daily request limit and whether 20,000 tickets per month would fit within it.
3. Second opinion (optional, +3 bonus): run the final prompt with a second model or provider and compare dev accuracy.

In [35]:
```
# Part D work here

# 1. Test-retest: a fresh client with the cache off, same settings as llm
fresh = LLM(PROVIDER, model=MODEL, min_interval=MIN_INTERVAL, max_retries=6, cache=False)

# Same steps as classify(), but with a chosen client
def classify_with(client, subject, body, system):
    raw = client.ask(f"Subject: {subject}\n\nBody: {body}", system=system,
                     json_mode=True, max_tokens=200, temperature=0)
    try:
        obj = llmkit.parse_json_object(raw)
    except ValueError:
        obj = {}
    if not isinstance(obj, dict):
        obj = {}
    return dict(category=validate_label(obj.get("category"), CATEGORIES),
                urgency=validate_label(obj.get("urgency"), URGENCY),
                sentiment=validate_label(obj.get("sentiment"), SENTIMENT))

# Run the final classifier (v3) twice on the dev set
run1 = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_with(fresh, r.subject, r.body, SYSTEM_V3)) for r
in dev_tickets.itertuples()])
run2 = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_with(fresh, r.subject, r.body, SYSTEM_V3)) for r
in dev_tickets.itertuples()])

# Compare the two runs field by field
both = run1.merge(run2, on="ticket_id", suffixes=("_run1", "_run2"))
fields = ["category", "urgency", "sentiment"]
same = pd.DataFrame({f: both[f + "_run1"] == both[f + "_run2"] for f in fields})
all_same = same.all(axis=1)
print(f"All three fields identical across runs: {all_same.sum()}/{len(both)} tickets ({all_same.mean():.0%})")
for f in fields:
    print(f"  {f}: {same[f].sum()}/{len(both)} identical")

# Dev accuracy of each run (shows how much accuracy moves by chance)
for name, run in [("run 1", run1), ("run 2", run2)]:
    m = dev.merge(run, on="ticket_id", suffixes=("_gold", "_pred"))
    accs = {f: round(float((m[f + "_gold"] == m[f + "_pred"]).mean()), 2) for f in fields}
    print(name, "accuracy:", accs)

# Tickets that changed between runs
changed = both[~all_same]
if len(changed):
    display(changed)

# 2. Cost: average tokens per ticket for v3, from the fresh client's usage log
```

```
calls = pd.DataFrame(fresh.calls)
avg_in = calls["input_tokens"].mean()
avg_out = calls["output_tokens"].mean()
print(f"\nCalls logged: {len(calls)}")
print(f"Average input tokens per ticket:  {avg_in:,.0f}")
print(f"Average output tokens per ticket: {avg_out:,.0f}")

# Google's published price for gemini-3.5-flash-lite (USD per 1M tokens)
PRICE_IN, PRICE_OUT = 0.30, 2.50
TICKETS_PER_MONTH = 20_000
cost_per_ticket = (avg_in * PRICE_IN + avg_out * PRICE_OUT) / 1_000_000
monthly_cost = TICKETS_PER_MONTH * cost_per_ticket
print(f"Cost per ticket: ${cost_per_ticket:.6f}")
print(f"Projected cost for {TICKETS_PER_MONTH:,} tickets/month: ${monthly_cost:,.2f}")

# Free-tier check: about 667 tickets per day (20,000 / 30 days)
FREE_RPM = 15          # Gemini 3.5 Flash Lite free tier, from Google AI Studio
FREE_RPD = 500         # Gemini 3.5 Flash Lite free tier, from Google AI Studio
per_day = TICKETS_PER_MONTH / 30
print(f"\nNeeded per day: about {per_day:,.0f} requests")
print(f"Ceiling from the per-minute limit alone: {FREE_RPM * 60 * 24:,} requests/day")
print(f"Free-tier daily limit: {FREE_RPD:,} requests/day -> fits: {per_day <= FREE_RPD}")
```

```
[llmkit] ServerError (attempt 1); waiting 2s before retrying.
All three fields identical across runs: 20/20 tickets (100%)
  category: 20/20 identical
  urgency: 20/20 identical
  sentiment: 20/20 identical
run 1 accuracy: {'category': 0.9, 'urgency': 0.85, 'sentiment': 0.95}
run 2 accuracy: {'category': 0.9, 'urgency': 0.85, 'sentiment': 0.95}

Calls logged: 40
Average input tokens per ticket:  1,209
Average output tokens per ticket: 28
Cost per ticket: $0.000433
Projected cost for 20,000 tickets/month: $8.66

Needed per day: about 667 requests
Ceiling from the per-minute limit alone: 21,600 requests/day
Free-tier daily limit: 500 requests/day -> fits: False
```

Part E. Predictions and recommendation (20 points)

1. Run your final pipeline on all 78 tickets and save `hw2_predictions.csv` with exactly these columns: `ticket_id, category, urgency, sentiment, invoice_ids, body_phones` (lists joined with `|`, sorted; empty if none).
2. Write a 200 to 300 word memo to the support manager: which fields can be automated now, which need human review, what the error patterns are, and what it will cost. Support every claim with a number from this notebook.

In [36]:
```
# Build `final` for all tickets, then:
# final[["ticket_id", "category", "urgency", "sentiment", "invoice_ids",
"body_phones"]].to_csv("hw2_predictions.csv", index=False)
# from google.colab import files; files.download("hw2_predictions.csv")
# Build `final` for all tickets, then:

# Classify all 78 tickets with the final prompt (v3); the 20 dev tickets come from the cache
labels = pd.DataFrame([dict(ticket_id=r.ticket_id, **classify_v3(r.subject, r.body)) for r in
tickets.itertuples()])
```

```python
# Extract invoice IDs and body phones (sorted, joined with |, empty if none)
final = tickets[["ticket_id", "body"]].merge(labels, on="ticket_id")
final["invoice_ids"] = final.body.map(lambda t: "|".join(extract_invoice_ids(t)))
final["body_phones"] = final.body.map(lambda t: "|".join(extract_phones(t)))

# Checks before saving
print("Rows:", len(final), "| unique ticket IDs:", final.ticket_id.is_unique)
print("Invalid labels:", final[["category", "urgency", "sentiment"]].isna().sum().to_dict())
print("Tickets with invoice IDs:", (final.invoice_ids != "").sum(), "| with body phones:", (final.body_phones
!= "").sum())

# Confirm the dev tickets still match the v3 results
m = dev.merge(final, on="ticket_id", suffixes=("_gold", "_pred"))
print("Dev accuracy:", {f: round(float((m[f + "_gold"] == m[f + "_pred"]).mean()), 2) for f in ["category",
"urgency", "sentiment"]})

# Label counts across all 78 tickets
for f in ["category", "urgency", "sentiment"]:
    print(f"\n{f}:", final[f].value_counts().to_dict())

# Save and download the predictions file
final[["ticket_id", "category", "urgency", "sentiment", "invoice_ids",
"body_phones"]].to_csv("hw2_predictions.csv", index=False)
from google.colab import files; files.download("hw2_predictions.csv")
display(final[["ticket_id", "category", "urgency", "sentiment", "invoice_ids", "body_phones"]].head(10))
```

```
Rows: 78 | unique ticket IDs: True
Invalid labels: {'category': 0, 'urgency': 0, 'sentiment': 0}
Tickets with invoice IDs: 7 | with body phones: 6
Dev accuracy: {'category': 0.9, 'urgency': 0.85, 'sentiment': 0.95}

category: {'Technical': 16, 'Integration': 14, 'Billing': 14, 'Feature Request': 13, 'Account Access': 12, 'Canc
ellation': 9}

urgency: {'Low': 31, 'Medium': 23, 'High': 18, 'Critical': 6}

sentiment: {'Neutral': 43, 'Negative': 22, 'Positive': 13}
```

	ticket_id	category	urgency	sentiment	invoice_ids	body_phones
0	T-1001	Integration	Low	Positive		
1	T-1002	Billing	Medium	Neutral	INV-2025-00417	+16055550142
2	T-1003	Cancellation	Medium	Neutral		
3	T-1004	Billing	Low	Neutral		
4	T-1005	Technical	Medium	Neutral		
5	T-1006	Feature Request	Low	Positive		
6	T-1007	Technical	Medium	Neutral	INV-2025-01022	
7	T-1008	Billing	High	Negative	INV-2025-00802	
8	T-1009	Account Access	Low	Neutral		
9	T-1010	Feature Request	Low	Neutral		

Memo: *your memo here*

Memo:

To: Support Manager, LedgerLoop

From: Sharon Wamalwa

Subject: Automated ticket triage: what is ready, what needs review, and cost

I tested an automated triage step on 20 labeled tickets, then ran it on all 78 tickets.

Automate now. Invoice IDs and phone numbers matched the correct answers on 100% of the 20 test tickets, and the classifier returned 0 invalid labels. Sentiment was correct on 95% (19/20) and category on 90% (18/20). These fields can be applied automatically, with occasional spot checks.

Keep human review for urgency. Urgency was correct on 85% (17/20). All High and Critical tickets were labeled correctly (3/3), but 2 of 7 Medium tickets were labeled Low, which could delay real problems. Agents should confirm urgency on tickets labeled Low or Medium. All 6 tickets labeled Critical in the full set describe a stopped business or a security issue, so those can go straight to the top of the queue.

Error patterns. The remaining 6 wrong fields came from three situations: a calm customer describing a real malfunction (urgency rated too low), a request to change broken behavior (labeled Feature Request instead of Technical), and a cancellation dated months ahead (urgency rated too high, because the model does not see the date the ticket arrived).

Reliability. Two identical runs gave the same labels on 20/20 tickets (100%), so results are repeatable.

Cost. Each ticket uses about 1,209 input and 28 output tokens, about $0.00043. For 20,000 tickets a month, that is about 8.66. The free tier allows 500 requests a day, but this volume needs about 667, so a paid plan is required.

Make your submission PDF (run this last)

Save your work first (File > Save), then run the cell below. It turns the whole notebook, outputs included, into `HW2_submission.pdf` and downloads it, plus an HTML copy as a backup. The first run takes one to two minutes because it installs a PDF printer.

Do not use File > Print in Colab: it prints only what is laid out on screen, so long notebooks are cut off partway down a page. If the PDF step fails, open the downloaded `HW2_submission.html` in your browser and print that page to PDF (Ctrl+P or Cmd+P, then *Save as PDF*); a standalone page prints in full.

In [ ]:
```
#@title Make the PDF and HTML copies of this notebook
# code hidden in this copy
```